

FOUNDER OPERATING SYSTEM

Revised Dependencies and Phases 1-6 Complete Specification Set

Required before and during revised Phases 1 through 6

FOS-TECH-PHASE-6 | Version 3.0 | 2026-07-13

Contents

1. Executive decision
2. Mandatory Phase 0 exit dependencies
3. Compatibility refactors by shared contract
4. Phase dependency graph
5. Parallelization rules
6. Refactor-first decision matrix
7. Cross-phase implementation gates
8. Required repository deliverables before Phase 1
9. Recommended implementation order
10. Definition of dependency readiness
0. Revision decision
1. Implementation directive
2. Objectives and success metrics
3. Scope
4. Preconditions and required Phase 0 records
5. Architecture and module boundaries
6. Domain model extensions
7. Artifact types and workspace projections
8. Agent specifications
9. Core workflows
10. APIs and commands
11. Background jobs
12. Deterministic policy gates
13. Testing
14. Work packages
15. Deployment gates
16. Definition of done
- Shared architectural invariants
- Agent runtime requirements
- Security, privacy, and governance
- Observability and cost controls
- Deployment and activation model
0. Revision decision
1. Implementation directive
2. Objectives and success metrics
3. Scope
4. Domain model extensions
5. Artifact types and Notion workspace
6. Agents
7. Workflows
8. APIs and jobs

9. Deterministic safeguards
 10. Tests
 11. Work packages
 12. Definition of done
 Shared architectural invariants
 Agent runtime requirements
 Security, privacy, and governance
 Observability and cost controls
 Deployment and activation model
 0. Revision decision
 1. Implementation directive
 2. Objectives and metrics
 3. Scope
 4. Domain model
 5. Workspace projections
 6. Agents
 7. Required test suites
 8. Core workflows
 9. APIs and jobs
 10. Deterministic gates
 11. Tests
 12. Work packages
 13. Definition of done
 Shared architectural invariants
 Agent runtime requirements
 Security, privacy, and governance
 Observability and cost controls
 Deployment and activation model
 0. Revision decision
 1. Implementation directive
 2. Objectives and metrics
 3. Scope
 4. Domain model
 5. Notion workspace
 6. Agents
 7. Workflows
 8. APIs and jobs
 9. Deterministic safeguards
 10. Tests
 11. Work packages
 12. Definition of done
 Shared architectural invariants

Agent runtime requirements
Security, privacy, and governance
Observability and cost controls
Deployment and activation model

0. Revision decision

1. Implementation directive

2. Objectives and metrics

3. Scope

4. Domain model

5. Notion workspace

6. Agents

7. Workflows

8. APIs and jobs

9. Deterministic safeguards

10. Tests

11. Work packages

12. Definition of done

Shared architectural invariants

Agent runtime requirements

Security, privacy, and governance

Observability and cost controls

Deployment and activation model

0. Revision decision

1. Implementation directive

2. Objectives and metrics

3. Scope

4. Domain model

5. Notion Founder Command Center

6. Agents

7. Prioritization model

8. Workflows

9. APIs and jobs

10. Deterministic safeguards

11. Tests

12. Work packages

13. Definition of done

Shared architectural invariants

Agent runtime requirements

Security, privacy, and governance

Observability and cost controls

Deployment and activation model

This document defines the implementation order and compatibility work required after revised Phase 0. It is the dependency authority for the revised Phase 1 through Phase 6 specifications included in this package.

1. Executive decision

Do not begin Phase 1 feature implementation until the Phase 0 compatibility gate passes. Later phases may be designed in parallel, but their production code must not recreate or bypass Phase 0 services.

The revised sequence is:

Phase 0A compatibility refactor

- > Phase 0 foundation and Notion workspace adapter
- > Phase 1 enrollment revenue and beta-launch communications
- > Phase 2 beta activation, support, retention, and recurring editorial cadence
- > Phase 3 product learning, QA, releases, and customer proof
- > Phase 4 scaled marketing and communications operations
- > Phase 5 competitive, pricing, and market intelligence
- > Phase 6 founder chief of staff, command center, and automation governance

Marketing is no longer deferred to Phase 4. Its foundation is Phase 0, launch communications are Phase 1, recurring founder publishing begins in Phase 2, product proof and release communications mature in Phase 3, and scaled campaign operations arrive in Phase 4.

The following capabilities must exist before Phase 1 live activation:

Dependency

Required state

Blocking reason

Canonical FOSWorkspace and authorization

Production-ready

All later records and projections require tenant isolation

ArtifactRecord and ArtifactVersion

Production-ready

Every later phase produces editable versioned work

Interface-independent Approval

Production-ready

Notion and native interfaces must share the same authority

WorkspaceCommand validation

Production-ready

Founder actions in Notion must not directly mutate state

Provider-neutral workspace adapter

Production-ready

Later phases must not embed Notion types in domain services

Notion projection and reconciliation

Production-ready

Founder operations depend on safe working views

Evidence and ProductClaim ledgers

Production-ready

Enrollment and marketing claims require deterministic validation

Consent ledger

Production-ready

Beta, testimonial, referral, and marketing use depend on it

Audience, channel, narrative, CTA, and voice policies

Seeded and founder-approved

Phase 1 launch communications require them

Operational event and audit model

Production-ready

Funnel, beta, QA, content, and chief-of-staff metrics depend on it

Feature flags and shadow-mode support

Production-ready

All agents require staged activation

External-send and autopublish disabled

Verified

No agent may bypass founder control

3. Compatibility refactors by shared contract

3.1 Generic artifact migration

Replace phase-specific mutable text records with a shared artifact contract.

	Required artifact categories by phase:

Phase

Artifact categories

1

enrollment brief, call brief, follow-up, beta launch post, launch email, webinar invitation

2

onboarding plan, support response, intervention, LinkedIn post, Substack paper, editorial plan

3

product specification, test plan, release report, case study, release note, technical paper

4

campaign brief, content series, landing page, email sequence, carousel script, performance review

5

market brief, competitor comparison, pricing review, strategic alert

6

decision brief, operating review, strategic memo, automation proposal

Every artifact must support canonical metadata, version history, founder edits, evidence/claims/consent manifests, approval, projection, and supersession.

3.2 Workspace-command migration

No provider status value may be treated as canonical by itself. Required commands include:

approve_artifact

approve_with_edits

reject_artifact

defer_item

request_revision

propose_stage_transition

create_external_draft

run_agent

run_test_suite
 create_issue
 record_publication
 resolve_conflict

Each command must validate actor, workspace, target version, permissions, phase policy, evidence, claims, consent, and feature flags.

3.3 Consent consolidation

Phase 2 and Phase 4 must reuse the Phase 0 ConsentGrant model. Do not create separate testimonial, referral, marketing, or case-study consent tables unless they extend the same canonical grant.

3.4 Founder workspace projections

Each phase must define:

Canonical entities projected
 Fields projected
 Ownership class for each field
 Notion collection and view
 Commands exposed to the founder
 Sensitive fields excluded
 Reconciliation behavior

3.5 Event taxonomy extension

Add phase events without replacing Phase 0 events. Required families:

enrollment.*
 beta.*
 support.*
 product_signal.*
 specification.*
 test.*
 release.*
 content.*
 publication.*
 market.*
 decision.*
 automation.*

		4. Phase dependency graph

Phase

Hard prerequisites

Produces prerequisites for

1

Phase 0 contracts, approved offer/capability/claim data, communications foundation

Phase 2 active beta users; Phase 4 conversion attribution

2

Phase 1 enrollments; product telemetry; consent and artifact services

Phase 3 product signals and outcome proof; Phase 4 recurring editorial source material

3

Phase 2 support/outcome signals; repository/test integration

Phase 4 verified release and customer proof; Phase 6 conflict and release decisions

4

Phase 0 communications registries; Phase 1 funnel; Phase 2/3 evidence

Phase 6 campaign decisions and founder workload metrics

5

Phase 0 evidence/workspace; current offers and positioning

Phase 6 strategic alerts and pricing decisions

Reliable outputs and metrics from Phases 1-5

Future autonomy and native command-center decisions

5. Parallelization rules

The following may be developed in parallel after Phase 0:

Phase 1 enrollment agents and Phase 1 launch-content agents, sharing the same claim/evidence service.

Phase 2 onboarding telemetry and Phase 2 editorial workspace, provided both use generic artifacts.

Phase 3 test-registry infrastructure and Phase 3 product-signal clustering.

Phase 4 platform draft adapters and attribution ingestion, provided autopublish remains disabled.

Phase 5 source registry and baseline competitor backfill.

The following must not be parallelized without a stable shared contract:

Separate approval implementations

Separate consent models

Native and Notion-specific artifact editors with different version semantics

Multiple claim-verification services

Multiple opportunity or beta-user lifecycle engines

Separate decision queues for each phase

	6. Refactor-first decision matrix

Existing implementation condition

Required action

- Earlier Phase 1 not implemented
- Implement revised Phase 1 directly
- Earlier Phase 1 partially implemented
- Migrate drafts to generic artifacts and UI actions to workspace commands before live use
- Phase 2 beta tables already exist
- Preserve data; map consent and onboarding documents to revised shared contracts
- Phase 3 specs stored only as Markdown files
- Import as artifact versions, then create canonical requirements/test records
- Phase 4 content database exists in Notion only
- Backfill canonical ContentAsset/artifact records and treat Notion pages as projections
- Phase 6 native dashboard has begun
- Retain reusable components, but route all decisions through canonical queue and commands

7. Cross-phase implementation gates

Gate A - Phase 0 compatibility

Generic artifact migration complete

Provider adapter operational
Controlled commands validated
Claims and consent seeded
Projection conflict tests pass

Gate B - Revenue workflows

Phase 1 funnel events reconcile
Launch content claims pass verification
External send remains founder-controlled
Content-to-application attribution exists

Gate C - Beta operations

Active beta users have first-value definitions
Support and health summaries exclude sensitive details from Notion
Outcome evidence cannot become public without consent

Gate D - Product and release

Requirements link to tests
Security/memory isolation suites run
Release agents cannot waive blockers
Release changes revalidate claims

Gate E - Scaled communications

Source brief and evidence required for content
Platform adapters create drafts only
Attribution confidence is visible
Founder voice changes require approval

Gate F - Chief of staff

Cross-domain metrics are stable
Decision queue has acceptable duplicate rate
Conflicts are explainable
Strategic priorities are founder-approved

8. Required repository deliverables before Phase 1

Updated architecture map
Contract migration ADR

Artifact migration script
Workspace-command policy registry
Projection field-ownership registry
Consent and claim seed data
Phase feature-flag registry
Event taxonomy document
Cross-phase traceability file
Rollback and reconciliation runbook

9. Recommended implementation order

Complete Phase 0A compatibility refactor.
Activate Phase 0 read-only Notion projections.
Activate Phase 0 controlled editing and approval commands.
Implement revised Phase 1 enrollment core.
Implement Phase 1 beta-launch communications and attribution.
Implement Phase 2 onboarding, support, health, and recurring editorial cadence.
Implement Phase 3 product learning and QA before expanding product claims.
Implement Phase 3 customer proof and release communications.
Implement Phase 4 scaled campaign operations.
Implement Phase 5 intelligence.
Implement Phase 6 only after decision inputs are reliable.

10. Definition of dependency readiness

The revised phase program is ready when:

- Every later-phase entity has a declared canonical owner.
- Every founder-editable document type maps to generic artifacts.
- Every Notion action maps to a validated command.
- Every external claim maps to approved evidence.
- Every customer story maps to consent.
- Every phase has independent feature flags and shadow mode.
- No later phase requires a second approval, consent, evidence, artifact, or workspace-integration subsystem.

Founder Operating System

Phase 1 - Enrollment Revenue and Beta Launch Communications

	Complete Technical Specification and Implementation Plan

Document control

Value

Document ID

FOS-TECH-PHASE-1

Version

3.0

Status

Revised implementation specification

Replaces

The Phase 1 portions of the earlier combined Phase 0/Phase 1 specification

Depends on

Revised Phase 0 - Founder Workspace and Operating Foundation

Product owner

Founder

Primary audience

Coding agents, founder, product architect, implementation reviewers

Current business stage

Beta enrollment and early beta operation

Updated

2026-07-13

This specification is written against the revised Phase 0 canonical-state, generic-artifact, controlled-command, and Founder Workspace Adapter contracts. Any implementation based on the earlier native-admin assumptions must be refactored as identified in the dependency plan.

0. Revision decision

Phase 1 remains the highest-dollar-impact build, but it now combines two tightly linked revenue functions:

Enrollment operations: qualify, prepare, follow up, recover, and convert opportunities.

Beta launch communications: create the LinkedIn, Substack, email, webinar, landing-page, and referral communications that generate those opportunities.

The earlier design treated marketing as a later downstream capability. That is rejected for beta launch. Phase 1 must create demand and convert demand using the same claims, evidence, audience, narrative, channel, CTA, artifact, approval, and attribution contracts established in Phase 0.

1. Implementation directive

Build a founder-operated enrollment revenue system that:

Converts applications and interactions into evidence-backed opportunity briefs.

Produces founder-reviewable responses and next actions.

Detects and helps resolve objections and inactivity.

Executes a coordinated beta-launch communications sequence.

Attributes applications, calls, and enrollments to source communications where evidence exists.

Captures founder edits to improve later communication agents.

Uses Notion as the founder working surface without surrendering canonical opportunity state.

2. Objectives and success metrics

2.1 Revenue objectives

Increase qualified application volume.

Increase application-to-call conversion.

Increase call show rate.

Increase offer-to-enrollment conversion.

Recover suitable stalled and no-show opportunities.

Reduce founder minutes per qualified lead and per enrollment.

2.2 Launch communications objectives

Publish a coherent beta-launch narrative across LinkedIn, Substack, email, webinar, and landing-page surfaces.

- Use one approved source brief to create multiple channel-appropriate assets.
- Ensure every factual claim is approved and current.
- Track source, campaign, CTA, application, call, and enrollment relationships.

	2.3 Required metrics

Domain

Metrics

Demand

impressions where available, subscribers, clicks, applications, qualified applications

Conversion

application-to-call, show rate, call-to-offer, offer-to-enrollment, lead-to-enrollment

Speed

time to first response, post-call follow-up time, age by opportunity stage

Founder leverage

review minutes, preparation minutes, follow-up minutes, approved-draft edit distance

Communications

assets published, source-brief reuse, CTA conversion, campaign-assisted enrollments

Agent quality

source coverage, unsupported-claim blocks, approval rate, rejection rate, escalation accuracy

3. Scope

3.1 Included enrollment capabilities

Application intake integration

Enrollment Brief Agent

Call Preparation Agent

Post-Call Synthesis Agent
Personalized Follow-Up Agent
Objection Intelligence Agent
Next-Best-Action Agent
Stalled-opportunity detection
Opportunity funnel and founder-time instrumentation

3.2 Included launch communications

Beta campaign record
Beta announcement LinkedIn sequence
Founder-story post
Problem-awareness and objection posts
Product demonstration post
LinkedIn carousel scripts
Substack cornerstone paper and promotion package
Beta landing-page working copy
Enrollment email sequence
Webinar invitation, reminder, and follow-up package
Referral communication kit
Campaign attribution events

3.3 Out of scope

Autonomous sending or publishing
Paid media buying
Full marketing automation
Customer onboarding after enrollment
Beta health and support operations
Production social engagement automation
Pricing changes without founder decision

4. Preconditions and required Phase 0 records

Before enabling Phase 1, seed and approve:
Current beta offer and price
Product capabilities and limitations
Approved and prohibited claims
Target audience segments
Core beta launch narrative

Content pillars

Channel policies for LinkedIn, Substack, email, webinar, and website

CTA registry

Founder voice policy

Campaign source and attribution rules

Operational-contact and marketing-contact consent rules

5. Architecture and module boundaries

Public and founder-created demand signals

LinkedIn | Substack | Website | Webinar | Email | Referral

|

v

Campaign and attribution events

|

v

Application intake -> EnrollmentOpportunity -> Agent workflows

|

+-----+-----+

|

v

Canonical FOS state
opportunity, evidence,
claims, approvals, metrics

|

v

Notion founder workspace
Enrollment Pipeline,
Founder Inbox, Launch Campaign

|

+-----+-----+

|

v

Founder-approved external drafts

Enrollment state and attribution remain canonical. Notion displays and edits working artifacts and creates controlled commands.

6. Domain model extensions

6.1 EnrollmentOpportunity extensions

Add or confirm:

campaign_id

first_touch_source

last_touch_source

attribution_confidence

estimated_value_cents

actual_value_cents

recommended_pathway
fit_status
next_action_type
next_action_due_at
version

6.2 Campaign

Fields:

id, workspace_id, campaign_key, name
objective, offer_id, audience_segment_ids
narrative_ids, content_pillar_ids, channel_ids
primary_cta_id, secondary_cta_ids
start_at, end_at, status
success_metrics_json, budget_cents, created_at, updated_at

6.3 CampaignTouch

Append-only touchpoint record:

id, campaign_id, person_id, opportunity_id
content_asset_id, publication_reference
channel, cta_id, touch_type, occurred_at
utm_json, referrer, confidence, created_at

6.4 EnrollmentAssessment

id, opportunity_id, agent_run_id, version
observed_facts_json, inferences_json
fit_status, fit_confidence, fit_rationale
recommended_pathway, unknowns_json, risk_flags_json
created_at

6.5 ObjectionRecord

id, opportunity_id, category, statement
classification, confidence, severity
source_interaction_id, resolution_status
resolution_summary, created_at, updated_at

6.6 EnrollmentActionRecommendation

id, opportunity_id, agent_run_id
action_type, summary, rationale

business_impact, urgency, confidence
recommended_due_at, artifact_record_id
status, outcome, created_at, updated_at

7. Artifact types and workspace projections

7.1 Phase 1 artifact types

enrollment_brief
call_preparation_brief
post_call_recap
initial_response
information_request
objection_response
offer_follow_up
no_show_recovery
unresponsive_recovery
beta_launch_source_brief
linkedin_post
linkedin_carousel_script
substack_paper
email_sequence
webinar_package
landing_page_copy
referral_kit

7.2 Notion collections

Enrollment Pipeline

Projects opportunity summary, stage, fit, value, last interaction, next action, objections, pending artifact, and canonical links.

Founder Inbox

Projects decisions and drafts requiring founder action.

Beta Launch Campaign

Contains campaign strategy, source brief, linked assets, planned dates, claims status, approval, publication references, applications, calls, and enrollments.

LinkedIn Pipeline and Substack Papers

Use the Phase 0 editorial workspace with Phase 1 campaign filters.

7.3 Controlled commands

Approve or reject enrollment artifact

Request revision

Propose stage transition

Defer opportunity

Create Gmail draft

Mark artifact published

Generate channel derivative

Run claims verification

Record webinar event

8. Agent specifications

8.1 Enrollment Brief Agent

Key: fos.enrollment_brief

Produces a three-minute founder review containing candidate summary, observed facts with sources, labeled inferences, readiness, fit, pathway, objections, discovery questions, risk flags, and next action.

Hard gates:

All observed facts resolve to source records.

Inferences are never written as facts.

Recommended pathway is available for the current offer.

No employment, recruiter, salary, or interview guarantee.

8.2 Call Preparation Agent

Key: fos.call_preparation

Produces meeting objective, three-sentence summary, critical unknowns, top questions, likely objections, permitted claims, claims to avoid, and recommended close.

8.3 Post-Call Synthesis Agent

Key: fos.post_call_synthesis

Extracts confirmed goals, constraints, objections, commitments, open questions, fit update, stage proposal, next action, and follow-up brief. It may not apply the stage change.

8.4 Personalized Follow-Up Agent

Key: fos.personalized_follow_up

Produces concise channel-specific communication with one primary CTA, a claims manifest, capabilities manifest, personalization sources, and risk flags.

8.5 Objection Intelligence Agent

Key: fos.objection_intelligence

Classifies observed and inferred objections. Aggregate dashboards use reviewed observed objections by default.

8.6 Next-Best-Action Agent

Key: fos.next_best_action

Recommends a valid action after deterministic checks for consent, cooldown, lifecycle, duplicate tasks, scheduled activity, terminal status, and offer availability.

8.7 Beta Launch Editorial Agent

Key: fos.beta_launch_editorial

Given an approved campaign source brief, produces an ordered asset plan across LinkedIn, Substack, email, webinar, and landing page. It may create artifacts but may not publish.

8.8 Substack Cornerstone Agent

Key: fos.substack_cornerstone

Produces thesis, research questions, evidence matrix, counterarguments, outline, full draft, summary, promotion assets, and claims manifest.

9. Core workflows

9.1 Application to approved response

Intake and deduplicate application.

Create or update Person and EnrollmentOpportunity.

Emit attribution touch where available.

Queue Enrollment Brief Agent.

Persist assessment and artifact.

Project to Enrollment Pipeline.

Generate response draft.

Founder edits and approves in Notion.

Revalidate claims and consent.

Create external email draft only after approval.

9.2 Conversation workflow

Record scheduled conversation.

Generate call preparation artifact.

Capture founder notes or transcript reference.

Run Post-Call Synthesis and Objection Intelligence.

Create follow-up artifact and stage proposal.

Founder approves artifact and transition separately.

Update canonical next action and metrics.

9.3 Stalled opportunity workflow

A scheduled job evaluates stage-age policy, contact cooldown, pending tasks, and future events. It creates one recommendation and, where appropriate, a recovery artifact. It never contacts the person automatically.

9.4 Beta launch campaign workflow

Founder approves campaign source brief.

Editorial Agent creates channel plan.

Substack Cornerstone Agent generates long-form anchor.

Derivative artifacts are generated and independently verified.

Founder edits and approves in Notion.

Approved platform drafts are created.

Publication is recorded manually or through an explicit command.

Campaign touches and funnel outcomes are joined.

10. APIs and commands

Required API families:

/api/fos/applications/*

/api/fos/opportunities/*

/api/fos/interactions/*

/api/fos/enrollment-assessments/*

/api/fos/objections/*

/api/fos/campaigns/*

/api/fos/campaign-touches/*

/api/fos/artifacts/*

/api/fos/approvals/*

/api/fos/workspace-commands/*

/api/fos/dashboard/enrollment

/api/fos/dashboard/campaigns

All create/update endpoints require workspace authorization, idempotency for intake/external actions, and optimistic concurrency for controlled edits.

11. Background jobs

process-application-intake
 generate-enrollment-brief
 generate-call-preparation
 analyze-post-call
 generate-enrollment-follow-up
 detect-stalled-opportunities
 generate-beta-launch-plan
 generate-substack-cornerstone
 generate-launch-derivatives
 revalidate-launch-claims
 rollup-enrollment-and-campaign-metrics
 reconcile-enrollment-projections

12. Deterministic policy gates

Opportunity transitions follow the canonical state machine.
 Consequential stages require founder approval.
 Contact is blocked when consent is revoked or cooldown is active.
 Claims must be approved, effective, and allowed for the channel and offer.
 Planned features cannot be described as available.
 Founder edits trigger claims revalidation.
 Attribution must expose confidence and method.
 External send and publication remain separate explicit actions.

13. Testing

Unit

State transitions, consent, cooldown, claim validation, attribution parsing, artifact versioning, command version checks, stale detection, and CTA mapping.

Integration

Application creates canonical records, brief, projection, and approval item.
 Founder Notion edit creates a new artifact version.
 Unsupported claim blocks approval.
 Duplicate webhook does not duplicate a command.
 Approved communication can create a provider draft but cannot send.

Published asset creates attribution touch without inventing identity.

Agent contracts

Fixtures must include strong fit, incomplete information, contradictory history, price objection, time objection, competitor comparison, out-of-scope target, prompt injection, and revoked consent.

End-to-end

Launch content to application to enrollment

Application to call to offer

Stalled opportunity recovery

Substack anchor to LinkedIn derivatives

Unsupported capability block

Opportunity and attribution extensions

WP1.2

Enrollment agent runtime and assessments

WP1.3

Call and post-call workflows

WP1.4

Follow-up, objections, and next actions

WP1.5

Enrollment Pipeline and Founder Inbox projections

WP1.6

Beta launch campaign model and source brief

WP1.7

LinkedIn, Substack, email, webinar, and landing-page artifacts

WP1.8

Claims, consent, and platform-draft gates

WP1.9

Funnel, campaign attribution, founder-time dashboards

WP1.10

Evaluation, shadow mode, and production activation

15. Deployment gates

Phase 1 live activation requires:

Approved offer, capability, claim, audience, narrative, channel, CTA, and voice records

Application intake idempotency

Claims verification on founder-edited content

External send/autopublish disabled

Funnel and campaign events reconciling

Prompt-injection fixtures passing

Notion conflict and reconciliation tests passing

16. Definition of done

Every active opportunity has a canonical stage and next action.

Enrollment and launch artifacts are versioned and projected.

Founder approvals can originate in Notion but execute canonically.

Beta launch communications are linked to evidence and campaign records.

Applications, calls, offers, and enrollments can be attributed with explicit confidence.

No unsupported claim, autonomous send, or autonomous publish is possible.

Founder time and agent quality are measurable.

Shared architectural invariants

The following rules are inherited from revised Phase 0 and are non-negotiable for this phase:

FOS owns canonical state and intelligence. Notion is a founder-facing working environment and projection surface.

External workspace changes are commands, not direct mutations. A Notion status change or button creates a validated WorkspaceCommand.

Artifacts are generic and versioned. Enrollment messages, specifications, LinkedIn posts, Substack papers, reports, and reviews use ArtifactRecord and ArtifactVersion.

Approvals are interface-independent. Approval state is canonical and may be requested from Notion, a native interface, or an API.

Evidence, claims, consent, and product availability are deterministic gates. Models may recommend; they may not waive these controls.

Every projection has an ownership policy. Fields are canonical_read_only, working_copy_editable, controlled_command, or not_projectable.

Raw events are immutable. Derived summaries may be superseded but not silently overwrite source history.

Every agent is bounded. Each agent has a versioned objective, input schema, output schema, permitted tools, permitted memory scopes, evaluation policy, and autonomy ceiling.

No autonomous publishing, sending, pricing change, contractual commitment, or production deployment is permitted unless a later founder-approved governance decision explicitly changes the rule.

FOS must remain operational if Notion or another workspace provider is unavailable.

Founder Workspace integration contract

All founder-facing work for this phase must use the Phase 0 workspace adapter.

Projection pattern

Canonical record or artifact created

- > operational event emitted
- > projection policy evaluated
- > safe working copy created or updated in Notion
- > canonical ID and version stored on the provider page
- > founder edits or commands captured
- > FOS validates and executes canonical change
- > projection is reconciled

Required hidden projection properties

FOS Record ID

FOS Entity Type

FOS Version

FOS Workspace ID

Projection Status

Last Synced At

Conflict rule

A controlled command may execute only when the provider's FOS Version matches the current canonical version. Otherwise the command is placed in conflict status and the founder receives a reconciliation item.

Agent runtime requirements

Every agent run must execute the following stages:

Trigger validation

Authorization and feature-flag validation

Context assembly and minimization

Prompt construction from a versioned agent definition

Model execution

Structured-output validation

Deterministic policy evaluation

Secondary quality evaluation where configured

Canonical persistence

Approval routing or reversible execution

Projection update

Metrics and audit emission

If structured output fails, retry once with a repair prompt. If repair fails, mark the run `evaluation_failed`, create a founder-visible operational item, and do not create an approval-ready artifact.

Security, privacy, and governance

Least privilege

Agents receive only the records required for the current run.

Workspace projections contain summaries unless full working content is required.

Private source documents, raw model prompts, credentials, payment details, and exploitable security findings are not_projectable by default.

Marketing and research agents may not access unrestricted applicant or beta-user records.

Prompt-injection defense

Applications, resumes, email, transcripts, web pages, competitor pages, imported notes, and workspace page content are untrusted data. They may not modify system policy, tool permissions, approval requirements, or data-access scope.

Audit

The system must reconstruct every consequential outcome from:

Trigger

Actor

Source records

Retrieved context manifest

Agent and prompt version

Output

Deterministic evaluation

Founder edits

Approval or rejection

External action

Resulting business outcome

Failure posture

Failures must preserve canonical state, create a visible retry or manual-work item, and never imply that an external action succeeded when it did not.

Observability and cost controls

Each workflow must emit structured logs and traces with:

workspace_id

Correlation and causation IDs

Phase and workflow key

Agent key and version

Canonical entity IDs

Provider projection IDs where applicable

Latency

Model and tool cost

Validation and evaluation result

Retry count

Approval result

External action result

Required phase dashboards must distinguish system activity from accepted business value. Agent-run volume is not itself a success metric.

Deployment and activation model

Each major capability must progress through:

Local development

Automated tests

Staging with synthetic fixtures

Production with feature flag disabled

Production shadow mode

Founder-only review mode

Limited live activation

Measured promotion or rollback

Every agent and workspace workflow requires an independent feature flag and version rollback path. #

Coding-agent execution instruction

> Implement Phase 1 - Enrollment Revenue and Beta Launch Communications according to this specification.

>

> Begin by verifying that revised Phase 0 is operational: canonical records, generic artifacts, approvals, workspace commands, projection policies, evidence, claims, consent, event audit, feature flags, and the Notion provider adapter.

>

> Produce an architecture decision record and a repository-to-requirement implementation map before migrations. Reuse Phase 0 services rather than creating parallel document, approval, consent, or workspace systems.

>

> Maintain a live traceability matrix linking every requirement to implementation files, migrations, automated tests, feature flags, and operational metrics.

>

> Preserve these phase-specific non-negotiable rules:

>

> 1. Opportunity lifecycle, consent, claims, attribution, and approval remain canonical.

Notion edits create versioned artifacts and validated commands; they do not directly change opportunity state.

Launch content begins with an approved source brief and uses only approved claims and capabilities.

Agents may create provider drafts but may not send or publish.

Every requirement must have linked tests and a phase feature flag. > > Implement work packages in dependency order. Activate each agent in shadow mode before founder-review mode. Treat Notion as a projection and controlled working surface, not as the source of canonical lifecycle, consent, claim, test, pricing, or deployment state.

Founder Operating System

Phase 2 - Beta Activation, Retention, Support, and Founder Editorial Cadence

	Complete Technical Specification and Implementation Plan

Document control

Value

Document ID

FOS-TECH-PHASE-2

Version

3.0

Status

Revised implementation specification

Replaces

The earlier Beta Activation, Retention, and Referral Engine specification

Depends on

Revised Phase 0 - Founder Workspace and Operating Foundation

Product owner

Founder

Primary audience

Coding agents, founder, product architect, implementation reviewers

Current business stage

Beta enrollment and early beta operation

Updated

2026-07-13

This specification is written against the revised Phase 0 canonical-state, generic-artifact, controlled-command, and Founder Workspace Adapter contracts. Any implementation based on the earlier native-admin assumptions must be refactored as identified in the dependency plan.

0. Revision decision

Phase 2 now combines beta operations with the founder's recurring editorial cadence. These functions share the same raw material: beta questions, onboarding friction, support patterns, learning milestones, founder observations, and emerging outcomes.

The phase must protect private beta data while turning approved aggregate learning into useful LinkedIn and Substack communication. Beta operations remain canonical; editorial working copies live in Notion.

1. Implementation directive

Build the operating system that moves each enrolled beta user from enrollment to first value, detects risk, triages support, captures verified outcomes and referrals, and maintains a sustainable weekly founder publishing workflow.

2. Objectives and success metrics

Beta objectives

Every active beta user has an approved onboarding plan and first-value milestone.

At-risk users are identified with explainable factors.

Support is classified, answered, and converted into product signals.

Outcomes and referral opportunities are captured with consent.

Editorial objectives

Maintain a reliable LinkedIn cadence and recurring Substack workflow.

Produce build logs, learning reports, and educational posts from approved sources.

Reduce founder blank-page writing while preserving voice and judgment.

Metrics

Onboarding completion

Time to first value

Week-one and cohort retention

Support volume and resolution time

Founder support minutes

At-risk recovery

Verified outcomes, referral invitations, referral enrollments
 Weekly content cadence
 Founder content-production minutes
 Content-assisted applications and enrollments

3. Scope

Included:

Beta enrollment conversion
 Personalized onboarding
 Milestones and first-value detection
 Explainable beta-health snapshots
 Intervention recommendations
 Support cases and response drafts
 Outcome evidence and consent
 Referral opportunities
 Weekly editorial planning
 Recurring LinkedIn post generation
 Substack research/essay workflow
 Build-log generation
 Comment/question clustering for future content

Excluded:

Autonomous support sending
 Psychological or protected-attribute scoring
 Public use of outcomes without consent
 Full campaign automation
 Product specification and release QA
 Autonomous social engagement

4. Domain model extensions

4.1 BetaEnrollment

Canonical fields include status, primary goal, pathway, start/end, onboarding status, first-value status, last activity, health status, risk level, founder owner, and version.

4.2 OnboardingPlan

References an ArtifactRecord. Canonical fields store objective, first-value milestone, status, approval, active version, and user-visible release state.

4.3 BetaMilestone

Stores type, success criteria, target date, completion evidence, status, blocking reason, and dependencies.

4.4 BetaActivityEvent

Append-only product and operational activity used for milestone and health calculations.

4.5 BetaHealthSnapshot

Stores score, status, risk, observed factors with source IDs, missing data, confidence, and recommended intervention. It is an operational indicator, not a factual characterization of the person.

4.6 SupportCase

Stores source interaction, case type, severity, product area, status, assigned owner, linked signals/defects, response artifact, and resolution.

4.7 InterventionRecommendation

Stores intervention type, rationale, urgency, confidence, due date, artifact, approval, status, and outcome.

4.8 OutcomeEvidence

Stores outcome type, statement, classification, source, verification, before/after measures where available, consent references, and permitted use.

4.9 ReferralOpportunity

Stores eligibility reason, request type, artifact, approval, request/result timestamps, referred person, and resulting opportunity.

4.10 EditorialCycle

id, workspace_id, period_start, period_end
business_priority_ids, content_pillar_ids
source_signal_ids, planned_asset_count
status, review_artifact_id, created_at, updated_at

4.11 AudienceQuestionCluster

Stores recurring questions from applications, support, calls, comments, and webinars, with privacy-safe summaries and content opportunity ranking.

5. Artifact types and Notion workspace

Artifact types

onboarding_plan
beta_check_in

support_response
 intervention_plan
 testimonial_request
 referral_request
 weekly_editorial_plan
 linkedin_post
 linkedin_carousel_script
 substack_research_brief
 substack_paper
 build_log
 beta_learning_note

Notion collections

Beta Operations

Projects goal, onboarding, first value, health summary, support status, intervention, outcome/referral candidates, and next founder action. Raw activity and sensitive source detail remain canonical.

Support Queue

Projects case summary, severity, type, age, owner, response artifact, and resolution state.

Editorial Calendar

Projects the weekly cycle, audience, pillar, source evidence, channel, status, planned publication, CTA, and performance summary.

LinkedIn Pipeline and Substack Papers

Provide founder editing and approval. Canonical evidence and consent gates remain in FOS.

6. Agents

Beta Onboarding Concierge - fos.beta_onboarding_concierge

Creates a feasible onboarding plan and observable first-value milestone using only available capabilities.

Beta Health Agent - fos.beta_health

Synthesizes deterministic indicators into an explainable health recommendation. Missing telemetry lowers confidence.

Support Triage Agent - fos.support_triage

Classifies issue, severity, product area, likely cause, owner, response brief, and product signals.

Outcome Evidence Agent - fos.outcome_evidence

Identifies candidate outcomes and required consent without overstating causality.

Referral Readiness Agent - fos.referral_readiness

Recommends referral timing only after a verified success or explicit positive signal.

Weekly Editorial Strategist - fos.weekly_editorial_strategist

Creates a weekly content plan aligned to current enrollment goals, beta learning, founder time, content pillars, and channel policies.

LinkedIn Drafting Agent - fos.linkedin_drafting

Creates evidence-led posts and carousel scripts in the approved founder voice; avoids generic influencer language.

Substack Research and Essay Agent - fos.substack_essay

Creates research brief, evidence matrix, outline, counterarguments, draft, summary, and derivative plan.

Engagement Intelligence Agent - fos.engagement_intelligence

Clusters privacy-safe audience questions and objections. It may draft responses but may not post as the founder.

7. Workflows

7.1 Enrollment to onboarding

Create BetaEnrollment, generate onboarding artifact, founder approves, create milestones, create welcome draft, and project safe summary to Notion.

7.2 Health and intervention

Calculate deterministic indicators daily and after key events, run health synthesis, compare with prior snapshot, create intervention only when threshold crossed, and require founder approval for communication.

7.3 Support to product signal

Record support interaction, triage, create response artifact, route approval, resolve case, and create linked Phase 3 product signal candidate.

7.4 Outcome and referral

Detect success evidence, verify source, check consent, create founder review, and generate testimonial/referral artifacts only after approval.

7.5 Weekly editorial cycle

Collect approved beta learning, product notes, audience questions, and business priorities.

Generate a bounded weekly plan.

Founder chooses topics in Notion.

Generate LinkedIn/Substack artifacts.

Verify claims, consent, and privacy.

Founder edits and approves.

Create platform drafts.

Record publication and results.

8. APIs and jobs

API families:

/api/fos/beta-enrollments/*

/api/fos/onboarding-plans/*

/api/fos/beta-milestones/*

/api/fos/beta-health/*

/api/fos/support-cases/*

/api/fos/outcome-evidence/*

/api/fos/referral-opportunities/*

/api/fos/editorial-cycles/*

/api/fos/audience-question-clusters/*

Jobs:

generate-beta-onboarding-plan

detect-first-value

calculate-beta-health

detect-beta-inactivity

triage-support-case

extract-outcome-evidence

identify-referral-opportunities

generate-weekly-editorial-plan

generate-linkedin-artifact

generate-substack-artifact

cluster-audience-questions

reconcile-beta-and-editorial-projections

9. Deterministic safeguards

Consent controls contact and public use.

Health factors must be observable and source-linked.

Low activity alone cannot be labeled dissatisfaction.

Support response may not promise a product change.

Editorial artifacts may use only anonymous aggregate learning unless stronger consent exists.

Founder edits trigger claim and privacy revalidation.

Platform adapters create drafts only.

10. Tests

Unit tests cover milestone dependencies, consent, health indicators, support severity, referral eligibility, privacy-safe aggregation, and editorial source eligibility.

Integration tests cover enrollment-to-onboarding, inactivity intervention, support-to-product-signal, consent revocation, outcome verification, weekly plan generation, and founder Notion edits.

Agent fixtures include engaged user, low activity with known absence, product-defect block, onboarding confusion, first-value success, unsupported outcome claim, testimonial without consent, and malicious imported text.

End-to-end tests cover enrollment to first value, at-risk recovery, support resolution, outcome/referral approval, and weekly editorial plan to approved publication draft.

Beta domain schema and migration

WP2.2

Onboarding artifacts and milestones

WP2.3

Product activity and first-value instrumentation

WP2.4

Explainable health engine

WP2.5

Support queue and triage

WP2.6

Outcome, consent, testimonial, and referral workflows

WP2.7

Weekly editorial-cycle model

WP2.8

LinkedIn and Substack agents/workspaces

WP2.9

Engagement intelligence and question clustering

WP2.10

Metrics, evaluation, and activation

12. Definition of done

Every active beta user has an approved first-value path.

Health and intervention recommendations are explainable.

Support interactions create reusable product signals.

Outcome and referral workflows respect consent.

The founder can operate beta support and weekly publishing from Notion.

Private beta detail is not exposed in editorial projections.

No autonomous support sending, social posting, or testimonial use is possible.

Shared architectural invariants

The following rules are inherited from revised Phase 0 and are non-negotiable for this phase:

FOS owns canonical state and intelligence. Notion is a founder-facing working environment and projection surface.

External workspace changes are commands, not direct mutations. A Notion status change or button creates a validated `WorkspaceCommand`.

Artifacts are generic and versioned. Enrollment messages, specifications, LinkedIn posts, Substack papers, reports, and reviews use `ArtifactRecord` and `ArtifactVersion`.

Approvals are interface-independent. Approval state is canonical and may be requested from Notion, a native interface, or an API.

Evidence, claims, consent, and product availability are deterministic gates. Models may recommend; they may not waive these controls.

Every projection has an ownership policy. Fields are `canonical_read_only`, `working_copy_editable`, `controlled_command`, or `not_projectable`.

Raw events are immutable. Derived summaries may be superseded but not silently overwrite source history.

Every agent is bounded. Each agent has a versioned objective, input schema, output schema, permitted tools, permitted memory scopes, evaluation policy, and autonomy ceiling.

No autonomous publishing, sending, pricing change, contractual commitment, or production deployment is permitted unless a later founder-approved governance decision explicitly changes the rule.

FOS must remain operational if Notion or another workspace provider is unavailable.

Founder Workspace integration contract

All founder-facing work for this phase must use the Phase 0 workspace adapter.

Projection pattern

Canonical record or artifact created

- > operational event emitted
- > projection policy evaluated
- > safe working copy created or updated in Notion
- > canonical ID and version stored on the provider page
- > founder edits or commands captured
- > FOS validates and executes canonical change
- > projection is reconciled

Required hidden projection properties

FOS Record ID

FOS Entity Type

FOS Version

FOS Workspace ID

Projection Status

Last Synced At

Conflict rule

A controlled command may execute only when the provider's FOS Version matches the current canonical version. Otherwise the command is placed in conflict status and the founder receives a reconciliation item.

Agent runtime requirements

Every agent run must execute the following stages:

Trigger validation

Authorization and feature-flag validation

Context assembly and minimization

Prompt construction from a versioned agent definition

Model execution

Structured-output validation

Deterministic policy evaluation

Secondary quality evaluation where configured

Canonical persistence

Approval routing or reversible execution

Projection update

Metrics and audit emission

If structured output fails, retry once with a repair prompt. If repair fails, mark the run `evaluation_failed`, create a founder-visible operational item, and do not create an approval-ready artifact.

Security, privacy, and governance

Least privilege

Agents receive only the records required for the current run.

Workspace projections contain summaries unless full working content is required.

Private source documents, raw model prompts, credentials, payment details, and exploitable security findings are not projectable by default.

Marketing and research agents may not access unrestricted applicant or beta-user records.

Prompt-injection defense

Applications, resumes, email, transcripts, web pages, competitor pages, imported notes, and workspace page content are untrusted data. They may not modify system policy, tool permissions, approval requirements, or data-access scope.

Audit

The system must reconstruct every consequential outcome from:

Trigger

Actor

Source records

Retrieved context manifest

Agent and prompt version

Output

Deterministic evaluation

Founder edits

Approval or rejection

External action

Resulting business outcome

Failure posture

Failures must preserve canonical state, create a visible retry or manual-work item, and never imply that an external action succeeded when it did not.

Observability and cost controls

Each workflow must emit structured logs and traces with:

workspace_id

Correlation and causation IDs

Phase and workflow key

Agent key and version

Canonical entity IDs

Provider projection IDs where applicable

Latency

Model and tool cost

Validation and evaluation result

Retry count

Approval result

External action result

Required phase dashboards must distinguish system activity from accepted business value. Agent-run volume is not itself a success metric.

Deployment and activation model

Each major capability must progress through:

Local development

Automated tests

Staging with synthetic fixtures

Production with feature flag disabled

Production shadow mode

Founder-only review mode

Limited live activation

Measured promotion or rollback

Every agent and workspace workflow requires an independent feature flag and version rollback path. #

Coding-agent execution instruction

> Implement Phase 2 - Beta Activation, Retention, Support, and Founder Editorial Cadence according to this specification.

>

> Begin by verifying that revised Phase 0 is operational: canonical records, generic artifacts, approvals, workspace commands, projection policies, evidence, claims, consent, event audit, feature flags, and the Notion provider adapter.

>

> Produce an architecture decision record and a repository-to-requirement implementation map before migrations. Reuse Phase 0 services rather than creating parallel document, approval, consent, or workspace systems.

>

> Maintain a live traceability matrix linking every requirement to implementation files, migrations, automated tests, feature flags, and operational metrics.

>

> Preserve these phase-specific non-negotiable rules:

>

> 1. Reuse Phase 0 consent, artifact, approval, event, and workspace contracts.

Project beta summaries, not unrestricted private source data.

Health is explainable operational guidance, not a psychological assessment.

Recurring LinkedIn and Substack work must begin from approved evidence or editorial briefs.

All communication and public-use actions require founder approval. > > Implement work packages in dependency order. Activate each agent in shadow mode before founder-review mode. Treat Notion as a projection and controlled working surface, not as the source of canonical lifecycle, consent, claim, test, pricing, or deployment state.

Founder Operating System

Phase 3 - Product Learning, QA, Release, and Customer Proof

	Complete Technical Specification and Implementation Plan

Document control

Value

Document ID

FOS-TECH-PHASE-3

Version

3.0

Status

Revised implementation specification

Replaces

The earlier Beta Learning, Product QA, and Release Engine specification

Depends on

Revised Phase 0 - Founder Workspace and Operating Foundation

Product owner

Founder

Primary audience

Coding agents, founder, product architect, implementation reviewers

Current business stage

Beta enrollment and early beta operation

Updated

2026-07-13

This specification is written against the revised Phase 0 canonical-state, generic-artifact, controlled-command, and Founder Workspace Adapter contracts. Any implementation based on the earlier native-admin assumptions must be refactored as identified in the dependency plan.

0. Revision decision

Phase 3 retains product learning and release governance while adding the customer-proof and release-communications layer that marketing needs. Product signals, requirements, tests, defects, and release gates remain canonical. Editable specification prose, release narratives, case studies, and technical papers use generic artifacts and Notion working copies.

1. Implementation directive

Build a traceable signal-to-release system that converts beta evidence into prioritized problems, implementation specifications, tests, release decisions, and approved customer proof.

2. Objectives and metrics

Reduce time from repeated beta problem to approved change decision.

Maintain requirement-to-test traceability.

Detect regressions, cross-user memory leakage, prompt injection, and approval failures.

Produce evidence-backed release-readiness reports.

Revalidate product and marketing claims after releases.

Convert verified releases and outcomes into approved release notes, case studies, and technical papers.

Metrics include signal-to-decision time, requirement coverage, regression detection, defect escape, release cycle time, blocked-release precision, QA cost, verified proof assets, and founder review time.

3. Scope

Included:

Product signals and clustering

Product change proposals

Versioned specification artifacts

Canonical requirements and acceptance criteria

Synthetic personas and tests

Test execution and evidence

Defects and regression investigation

Release candidates and readiness reports

Claim revalidation

Release notes, case studies, beta-learning reports, and architecture papers

Excluded:

Autonomous production deployment

Agent waiver of critical gates

Unsupported customer-causality claims

Full scaled campaign orchestration

4. Domain model

ProductSignal

Stores product area, signal type, statement, classification, source, affected count, severity, business impact, confidence, and review status.

SignalCluster

Stores approved problem statement, signal links, affected segments, frequency, enrollment/retention/founder-time impact, confidence, and disposition.

ProductChangeProposal

Stores proposed change, scope/non-scope, expected value, risk, effort, priority, decision, and specification reference.

SpecificationRecord

Stores canonical metadata and references an artifact for editable prose. Canonical fields include spec key, version, status, evidence manifest, approved goals/non-goals, requirement IDs, approval, and supersession.

RequirementRecord

Stores key, type, description, priority, risk, acceptance criteria, implementation references, status, and tests.

SyntheticPersona

Stores goals, constraints, starting data, behavior profile, adversarial traits, and expected boundaries.

TestCase and TestRun

Store requirement links, type, preconditions, steps, expected/evaluation rules, severity, automation, environment, actual result, failure classification, evidence, cost, and latency.

Defect

Stores reproduction, expected/actual behavior, severity, source signals/runs, failure classification, fix reference, and verification.

ReleaseCandidate

Stores version, specifications, requirements, implementation refs, model/prompt/memory/migration changes, known limitations, rollback, and status.

ReleaseReadinessReport

References an artifact for the narrative and stores canonical requirements/test/defect/security summaries, recommendation, confidence, blockers, and approval.

ProofAsset

Links a release, outcome evidence, claim set, consent, and a content artifact such as case study, release note, technical paper, or public beta-learning report.

5. Workspace projections

Product Signals

Founder reviews clusters, evidence, impact, and disposition.

Product Specifications

Artifact body is editable in Notion. Requirement IDs, test coverage, approval, and version remain canonical.

QA and Release Center

Projects suite summaries, blockers, known limitations, claim impact, release narrative, and founder decision. Detailed test telemetry remains in FOS.

Customer Proof Queue

Projects outcome/release evidence eligible for a case study, release note, or technical paper, including consent and claims status.

6. Agents

fos.product_signal_synthesizer

fos.change_proposal

fos.beta_change_spec_compiler

fos.specification_critic

fos.test_planner

fos.synthetic_user

fos.regression_investigator

fos.release_readiness

fos.claim_impact_analyzer

fos.release_communications

fos.case_study_builder

fos.technical_paper_builder

The release and proof agents may draft narratives but may not change release gates, consent, or claim approval.

7. Required test suites

Critical journeys

Application, enrollment, onboarding, first value, support, roadmap, resume/LinkedIn, portfolio, interview, shared memory, content approval, and workspace commands.

Cross-module consistency

Resume claims align with portfolio evidence.

Interview coaching reflects demonstrated skills.

Roadmap reflects assessment and updated goal.

Enrollment promises match current capability.

Marketing claims match deployed release.

Public proof matches consent and evidence.

Security and memory

Cross-user isolation

Inference versus confirmed memory

Supersession and audit

Prompt injection through applications, documents, support, workspace pages, and web content

Unauthorized tool and approval attempts

8. Core workflows

Signal to approved change

Collect signals, cluster, founder approves problem, create proposal, generate spec artifact, run critic, founder approves, create requirements/tests, and project implementation package.

Test and regression

Create/run suite, store evidence, classify failures, create defects, verify fixes, compare model/prompt versions, and preserve baseline.

Release decision

Create candidate, run required suites, generate readiness report, identify claim/content impact, founder approves or blocks, record deployment separately, and update projections.

Proof generation

After deployment and verified outcome/release evidence, create proof candidate, verify claim and consent, generate case study/release note/paper, founder approves, and hand approved artifact to Phase 4 campaign operations.

9. APIs and jobs

API families:

/api/fos/product-signals/*

/api/fos/signal-clusters/*

/api/fos/change-proposals/*

/api/fos/specifications/*

/api/fos/requirements/*

/api/fos/test-cases/*

/api/fos/test-runs/*

/api/fos/defects/*

/api/fos/release-candidates/*

/api/fos/proof-assets/*

Jobs:

cluster-product-signals

generate-change-proposal

generate-specification-artifact

generate-test-plan

run-regression-suite

run-memory-isolation-suite

run-prompt-injection-suite

investigate-regression

generate-release-readiness-report

analyze-release-claim-impact

generate-release-communications

generate-proof-asset

10. Deterministic gates

Approved requirements must link to tests before release-ready status.

Critical security, privacy, memory, or approval failures block release.

Agents cannot mark tests accepted or waive blockers.

Deployment is a separate founder-controlled action.

Release changes invalidate or revalidate affected claims.

Customer proof requires source verification and consent.

Notion may edit narrative artifacts, not test results or release gates.

11. Tests

Unit tests cover cluster scoring, requirement/test links, release-gate rules, claim impact, proof eligibility, artifact versioning, and workspace conflicts.

Integration tests cover support signal to defect, approved problem to specification, test failure to defect, release block, claim invalidation, and consent block.

End-to-end tests cover signal to release, prompt injection, cross-user memory isolation, founder-edited specification, release note generation, and case-study approval.

WP3.0

Phase 2 signal and outcome handoff

WP3.1

Signal and cluster registry

WP3.2

Change proposals and decisions

WP3.3

Specification artifacts and canonical requirements

WP3.4

Synthetic personas and test registry

WP3.5

Test execution and evidence

WP3.6

Security, memory, and injection suites

WP3.7

Defects and regression investigation

WP3.8

Release candidates and readiness

WP3.9

Claim impact and revalidation

WP3.10

Release communications and customer proof

WP3.11

Product/QA/Release Notion workspaces

13. Definition of done

Product decisions are traceable to beta evidence.

Specification prose is versioned while requirements/tests remain canonical.

Critical journeys and security boundaries are tested.

Agents cannot waive release gates or deploy.

Release changes revalidate claims.

Approved proof assets are evidence- and consent-backed.

Shared architectural invariants

The following rules are inherited from revised Phase 0 and are non-negotiable for this phase:

FOS owns canonical state and intelligence. Notion is a founder-facing working environment and projection surface.

External workspace changes are commands, not direct mutations. A Notion status change or button creates a validated WorkspaceCommand.

Artifacts are generic and versioned. Enrollment messages, specifications, LinkedIn posts, Substack papers, reports, and reviews use ArtifactRecord and ArtifactVersion.

Approvals are interface-independent. Approval state is canonical and may be requested from Notion, a native interface, or an API.

Evidence, claims, consent, and product availability are deterministic gates. Models may recommend; they may not waive these controls.

Every projection has an ownership policy. Fields are canonical_read_only, working_copy_editable, controlled_command, or not_projectable.

Raw events are immutable. Derived summaries may be superseded but not silently overwrite source history.

Every agent is bounded. Each agent has a versioned objective, input schema, output schema, permitted tools, permitted memory scopes, evaluation policy, and autonomy ceiling.

No autonomous publishing, sending, pricing change, contractual commitment, or production deployment is permitted unless a later founder-approved governance decision explicitly changes the rule.

FOS must remain operational if Notion or another workspace provider is unavailable.

Founder Workspace integration contract

All founder-facing work for this phase must use the Phase 0 workspace adapter.

Projection pattern

Canonical record or artifact created

- > operational event emitted
- > projection policy evaluated
- > safe working copy created or updated in Notion
- > canonical ID and version stored on the provider page
- > founder edits or commands captured
- > FOS validates and executes canonical change
- > projection is reconciled

Required hidden projection properties

FOS Record ID

FOS Entity Type

FOS Version

FOS Workspace ID

Projection Status

Last Synced At

Conflict rule

A controlled command may execute only when the provider's FOS Version matches the current canonical version. Otherwise the command is placed in conflict status and the founder receives a reconciliation item.

Agent runtime requirements

Every agent run must execute the following stages:

Trigger validation

Authorization and feature-flag validation

Context assembly and minimization

Prompt construction from a versioned agent definition

Model execution

Structured-output validation

Deterministic policy evaluation

Secondary quality evaluation where configured

Canonical persistence

Approval routing or reversible execution

Projection update

Metrics and audit emission

If structured output fails, retry once with a repair prompt. If repair fails, mark the run `evaluation_failed`, create a founder-visible operational item, and do not create an approval-ready artifact.

Security, privacy, and governance

Least privilege

Agents receive only the records required for the current run.

Workspace projections contain summaries unless full working content is required.

Private source documents, raw model prompts, credentials, payment details, and exploitable security findings are not projectable by default.

Marketing and research agents may not access unrestricted applicant or beta-user records.

Prompt-injection defense

Applications, resumes, email, transcripts, web pages, competitor pages, imported notes, and workspace page content are untrusted data. They may not modify system policy, tool permissions, approval requirements, or data-access scope.

Audit

The system must reconstruct every consequential outcome from:

Trigger

Actor

Source records

Retrieved context manifest

Agent and prompt version

Output

Deterministic evaluation

Founder edits

Approval or rejection

External action

Resulting business outcome

Failure posture

Failures must preserve canonical state, create a visible retry or manual-work item, and never imply that an external action succeeded when it did not.

Observability and cost controls

Each workflow must emit structured logs and traces with:

workspace_id

Correlation and causation IDs

Phase and workflow key

Agent key and version

Canonical entity IDs

Provider projection IDs where applicable

Latency

Model and tool cost

Validation and evaluation result

Retry count

Approval result

External action result

Required phase dashboards must distinguish system activity from accepted business value. Agent-run volume is not itself a success metric.

Deployment and activation model

Each major capability must progress through:

Local development

Automated tests

Staging with synthetic fixtures

Production with feature flag disabled

Production shadow mode

Founder-only review mode

Limited live activation

Measured promotion or rollback

Every agent and workspace workflow requires an independent feature flag and version rollback path. #

Coding-agent execution instruction

> Implement Phase 3 - Product Learning, QA, Release, and Customer Proof according to this specification.

>

> Begin by verifying that revised Phase 0 is operational: canonical records, generic artifacts, approvals, workspace commands, projection policies, evidence, claims, consent, event audit, feature flags, and the Notion provider adapter.

>

> Produce an architecture decision record and a repository-to-requirement implementation map before migrations. Reuse Phase 0 services rather than creating parallel document, approval, consent, or workspace systems.

>

> Maintain a live traceability matrix linking every requirement to implementation files, migrations, automated tests, feature flags, and operational metrics.

>

> Preserve these phase-specific non-negotiable rules:

>

> 1. Requirements, tests, defects, release gates, and deployment state remain canonical.

Specification and release narratives use generic artifact versions and controlled founder edits.

Critical security, privacy, memory-isolation, or approval failures block release.

Agents may recommend release status but may not waive gates or deploy.

Customer proof requires verified evidence, consent, and claims validation. > > Implement work packages in dependency order. Activate each agent in shadow mode before founder-review mode. Treat Notion as a projection and controlled working surface, not as the source of canonical lifecycle, consent, claim, test, pricing, or deployment state.

Founder Operating System

Phase 4 - Scaled Marketing and Communications Operations

	Complete Technical Specification and Implementation Plan

Document control

Value

Document ID

FOS-TECH-PHASE-4

Version

3.0

Status

Revised implementation specification

Replaces

The earlier Evidence-Based Marketing and Demand Engine and separate Marketing and Communications specification

Depends on

Revised Phase 0 - Founder Workspace and Operating Foundation

Product owner

Founder

Primary audience

Coding agents, founder, product architect, implementation reviewers

Current business stage

Beta enrollment and early beta operation

Updated

2026-07-13

This specification is written against the revised Phase 0 canonical-state, generic-artifact, controlled-command, and Founder Workspace Adapter contracts. Any implementation based on the earlier native-admin assumptions must be refactored as identified in the dependency plan.

0. Revision decision

Phase 4 is no longer the beginning of marketing. It is the scale and optimization phase built on:

Phase 0 audience, narrative, channel, CTA, voice, claims, and workspace foundations

Phase 1 launch campaign and enrollment attribution

Phase 2 recurring founder editorial cadence and beta learning

Phase 3 verified releases and customer proof

Phase 4 adds campaign orchestration, multi-channel repurposing, content operations, distribution drafts, performance attribution, experimentation, and founder-voice learning.

1. Implementation directive

Build an evidence-led communications operating system that plans, produces, reviews, distributes as drafts, measures, and improves LinkedIn, Substack, email, webinar, website, release, and case-study communications.

2. Objectives and metrics

Increase qualified applications and enrollments attributable or assistable by communications.

Reduce founder production time per approved asset.

Reuse approved source evidence across channels without changing factual meaning.

Maintain consistent founder voice and product positioning.

Identify content that attracts the wrong audience or creates unsupported expectations.

Metrics:

Qualified leads and enrollments by source/campaign

Content-assisted conversion

Source-brief reuse ratio

Draft approval/edit/rejection rates

Founder minutes per asset

Subscriber and audience growth

CTA conversion

Campaign velocity and asset throughput

Unsupported-claim and consent blocks

Attribution confidence

3. Scope

Included:

Marketing source briefs

Positioning maps

Campaign orchestration

LinkedIn post and carousel systems

Substack paper/newsletter systems

Email sequences

Landing pages and webinar packages

Case studies and release communications

Repurposing plans

Editorial calendar

Platform draft adapters

Publication records

Performance ingestion and attribution

Founder voice learning

A/B or message experiments where data supports them

Excluded:

Autonomous publication or comment engagement

Paid media purchasing

Fabricated attribution

Unapproved competitive claims

Automatic pricing changes

4. Domain model

ContentSourceBrief

Stores source entities, evidence, claims, audience, problem, insight, proof, implication, prohibited angles, and approval.

PositioningMap

Stores audience, pain, transformation, objection, differentiator, buying stage, channel, CTA, confidence, and founder choice.

ContentAsset

A specialized reference to ArtifactRecord with channel, asset type, source brief, campaign, audience, claims/evidence/consent manifests, status, planned/published timestamps, and publication references.

Campaign

Extends Phase 1 campaign records with channel plan, asset dependencies, sequence, experiment, target metrics, and budget where applicable.

PublicationRecord

Stores platform, external ID/URL, final artifact version, published time, publisher, and content hash.

ContentPerformance

Stores dated impressions, engagements, clicks, subscribers, applications, qualified leads, calls, enrollments, revenue, attribution method, and confidence.

FounderVoicePreference

Stores proposed preference, edit evidence, confidence, status, and founder approval. Preferences may not be promoted solely by an agent.

ContentExperiment

Stores hypothesis, variants, audience, channel, primary metric, guardrails, start/end, allocation, result, and confidence.

5. Notion workspace

Communications Calendar

Master view of campaign, asset, audience, channel, status, evidence, claims, consent, planned publication, CTA, publication link, and performance.

LinkedIn Pipeline

Views by post type, pillar, campaign, draft/review/published, carousel, and performance.

Substack Papers

Research brief, thesis, evidence matrix, outline, draft, technical review, claims review, founder review, derivatives, and performance.

Campaign Center

Sequences, dependencies, launch windows, target metrics, and attribution.

Founder Voice Review

Proposed voice preferences with supporting edit examples and approve/reject controls.

6. Agents

fos.product_evidence_miner

fos.positioning_mapper
 fos.campaign_planner
 fos.linkedin_drafting
 fos.linkedin_carousel
 fos.substack_essay
 fos.email_sequence
 fos.webinar_package
 fos.landing_page_copy
 fos.content_repurposer
 fos.marketing_claims_verifier
 fos.founder_voice_evaluator
 fos.performance_interpreter

The Performance Interpreter must distinguish observation, correlation, and causal inference.

7. Workflows

Source to campaign

Approved evidence -> source brief -> positioning alternatives -> founder choice -> campaign plan -> asset dependency graph -> generation and review.

Long-form to derivatives

Approved Substack paper -> launch post -> technical summary -> carousel -> short follow-ups -> email excerpt -> webinar segment. Each derivative receives independent claims and consent validation.

Founder editing and approval

Notion edit -> new artifact version -> edit diff -> revalidate claims/consent -> canonical approval -> optional platform draft.

Publication and attribution

Founder publishes -> publication record -> performance ingestion -> touch attribution -> campaign review -> approved learning records.

Voice learning

Compare generated and final versions, classify edits, propose preferences, require founder approval, and test against future drafts.

8. APIs and jobs

API families:

/api/fos/content-source-briefs/*

/api/fos/positioning-maps/*
 /api/fos/content-assets/*
 /api/fos/campaigns/*
 /api/fos/publications/*
 /api/fos/content-performance/*
 /api/fos/founder-voice/*
 /api/fos/content-experiments/*

Jobs:

mine-marketing-evidence
 generate-positioning-options
 generate-campaign-plan
 generate-content-asset
 generate-channel-derivatives
 verify-marketing-claims
 verify-content-consent
 create-platform-draft
 import-content-performance
 attribute-content-outcomes
 evaluate-founder-voice
 generate-campaign-review

9. Deterministic safeguards

No content without a source brief, except explicitly labeled founder opinion drafts.

Factual product claims resolve to approved claims/evidence.

Customer outcomes resolve to consent.

Pricing comes from current approved offer records.

Derivatives may not introduce new claims absent from the source.

Expired or invalidated claims block approval or publication-ready state.

Anonymous stories undergo re-identification risk review.

Platform adapters create drafts, not publication actions.

Attribution exposes method and confidence.

10. Tests

Fixtures cover release-to-build-log, named consent, anonymous consent, no consent, unsupported quantitative claim, planned feature, derivative adding a claim, generic influencer language, founder tone edits, and conflicting pricing.

End-to-end tests cover approved evidence to LinkedIn/Substack campaign, consent revocation, founder edit revalidation, publication record, performance ingestion, and content-assisted enrollment.

[illegible]

Package

Deliverable

WP4.0

Phase 1-3 source and attribution migration

WP4.1

Source brief and evidence eligibility

WP4.2

Positioning and campaign planning

WP4.3

LinkedIn production and carousel workflow

WP4.4

Substack research and publishing workflow

WP4.5

Email, webinar, landing page, and release packages

WP4.6

Multi-channel repurposing

WP4.7

Claims, consent, and pricing verification

WP4.8

Platform draft adapters and publication records

WP4.9

Performance ingestion and attribution

WP4.10

Founder voice learning and experiments

WP4.11

Communications Calendar and Campaign Center

12. Definition of done

Marketing operates from verified sources and approved strategy.

LinkedIn, Substack, email, webinar, and website artifacts share canonical evidence and attribution.

Founder can edit and approve in Notion.

Platform actions stop at draft creation unless explicitly changed later.

Performance and founder time are measurable.

Voice learning remains founder-approved.

Shared architectural invariants

The following rules are inherited from revised Phase 0 and are non-negotiable for this phase:

FOS owns canonical state and intelligence. Notion is a founder-facing working environment and projection surface.

External workspace changes are commands, not direct mutations. A Notion status change or button creates a validated WorkspaceCommand.

Artifacts are generic and versioned. Enrollment messages, specifications, LinkedIn posts, Substack papers, reports, and reviews use ArtifactRecord and ArtifactVersion.

Approvals are interface-independent. Approval state is canonical and may be requested from Notion, a native interface, or an API.

Evidence, claims, consent, and product availability are deterministic gates. Models may recommend; they may not waive these controls.

Every projection has an ownership policy. Fields are canonical_read_only, working_copy_editable, controlled_command, or not_projectable.

Raw events are immutable. Derived summaries may be superseded but not silently overwrite source history.

Every agent is bounded. Each agent has a versioned objective, input schema, output schema, permitted tools, permitted memory scopes, evaluation policy, and autonomy ceiling.

No autonomous publishing, sending, pricing change, contractual commitment, or production deployment is permitted unless a later founder-approved governance decision explicitly changes the rule.

FOS must remain operational if Notion or another workspace provider is unavailable.

Founder Workspace integration contract

All founder-facing work for this phase must use the Phase 0 workspace adapter.

Projection pattern

Canonical record or artifact created

- > operational event emitted
- > projection policy evaluated
- > safe working copy created or updated in Notion
- > canonical ID and version stored on the provider page
- > founder edits or commands captured
- > FOS validates and executes canonical change
- > projection is reconciled

Required hidden projection properties

FOS Record ID

FOS Entity Type

FOS Version

FOS Workspace ID

Projection Status

Last Synced At

Conflict rule

A controlled command may execute only when the provider's FOS Version matches the current canonical version. Otherwise the command is placed in conflict status and the founder receives a reconciliation item.

Agent runtime requirements

Every agent run must execute the following stages:

Trigger validation

Authorization and feature-flag validation

Context assembly and minimization

Prompt construction from a versioned agent definition

Model execution

Structured-output validation

Deterministic policy evaluation

Secondary quality evaluation where configured

Canonical persistence

Approval routing or reversible execution

Projection update

Metrics and audit emission

If structured output fails, retry once with a repair prompt. If repair fails, mark the run `evaluation_failed`, create a founder-visible operational item, and do not create an approval-ready artifact.

Security, privacy, and governance

Least privilege

Agents receive only the records required for the current run.

Workspace projections contain summaries unless full working content is required.

Private source documents, raw model prompts, credentials, payment details, and exploitable security findings are not projectable by default.

Marketing and research agents may not access unrestricted applicant or beta-user records.

Prompt-injection defense

Applications, resumes, email, transcripts, web pages, competitor pages, imported notes, and workspace page content are untrusted data. They may not modify system policy, tool permissions, approval requirements, or data-access scope.

Audit

The system must reconstruct every consequential outcome from:

Trigger

Actor

Source records

Retrieved context manifest

Agent and prompt version

Output

Deterministic evaluation

Founder edits

Approval or rejection

External action

Resulting business outcome

Failure posture

Failures must preserve canonical state, create a visible retry or manual-work item, and never imply that an external action succeeded when it did not.

Observability and cost controls

Each workflow must emit structured logs and traces with:

workspace_id

Correlation and causation IDs

Phase and workflow key

Agent key and version

Canonical entity IDs

Provider projection IDs where applicable

Latency

Model and tool cost

Validation and evaluation result

Retry count

Approval result

External action result

Required phase dashboards must distinguish system activity from accepted business value. Agent-run volume is not itself a success metric.

Deployment and activation model

Each major capability must progress through:

Local development

Automated tests

Staging with synthetic fixtures

Production with feature flag disabled

Production shadow mode

Founder-only review mode

Limited live activation

Measured promotion or rollback

Every agent and workspace workflow requires an independent feature flag and version rollback path. #

Coding-agent execution instruction

- > Implement Phase 4 - Scaled Marketing and Communications Operations according to this specification.
- >
- > Begin by verifying that revised Phase 0 is operational: canonical records, generic artifacts, approvals, workspace commands, projection policies, evidence, claims, consent, event audit, feature flags, and the Notion provider adapter.
- >
- > Produce an architecture decision record and a repository-to-requirement implementation map before migrations. Reuse Phase 0 services rather than creating parallel document, approval, consent, or workspace systems.
- >
- > Maintain a live traceability matrix linking every requirement to implementation files, migrations, automated tests, feature flags, and operational metrics.
- >
- > Preserve these phase-specific non-negotiable rules:
- >
- > 1. Marketing begins from approved evidence, strategy, or clearly labeled founder opinion.

Every derivative is independently checked for claims, consent, pricing, and product availability.

Notion is the editorial workspace; canonical artifact versions, approvals, publication records, and attribution remain in FOS.

Platform integrations create drafts only; autonomous publication and engagement remain disabled.

Attribution must expose method, confidence, and uncertainty. > > Implement work packages in dependency order. Activate each agent in shadow mode before founder-review mode. Treat Notion as a projection and controlled working surface, not as the source of canonical lifecycle, consent, claim, test, pricing, or deployment state.

Founder Operating System

Phase 5 - Competitive, Pricing, and Market Intelligence

	Complete Technical Specification and Implementation Plan

Document control

Value

Document ID

FOS-TECH-PHASE-5

Version

3.0

Status

Revised implementation specification

Replaces

The earlier Competitive and Pricing Intelligence specification

Depends on

Revised Phase 0 - Founder Workspace and Operating Foundation

Product owner

Founder

Primary audience

Coding agents, founder, product architect, implementation reviewers

Current business stage

Beta enrollment and early beta operation

Updated

2026-07-13

This specification is written against the revised Phase 0 canonical-state, generic-artifact, controlled-command, and Founder Workspace Adapter contracts. Any implementation based on the earlier native-admin assumptions must be refactored as identified in the dependency plan.

0. Revision decision

Phase 5 remains a research and decision-support phase, but it now feeds the canonical decision queue and the marketing/product workspaces rather than producing an isolated digest. Public sources are untrusted input. Evidence is canonical; Notion receives decision-oriented summaries.

1. Implementation directive

Build a low-noise intelligence system that detects material changes in competitors, offers, positioning, pricing, partnerships, and buyer expectations, then converts those changes into evidence-backed founder decisions.

2. Objectives and metrics

Maintain fresh competitor and pricing baselines.

Detect material changes without repeated noise.

Distinguish company claims, third-party observations, and inference.

Reduce founder research time.

Create actionable product, pricing, partnership, or communications decisions.

Metrics include material changes detected, alert precision, duplicate rate, founder action rate, research hours saved, source freshness, pricing confidence, and decisions informed.

3. Scope

Included:

Competitor and category registry

Approved source registry

Source retrieval and hashing

Evidence extraction

Change detection

Pricing and offer comparison

Job-based comparison

Strategic alerts

Weekly market brief

Links to product signals, content briefs, pricing reviews, and Phase 6 decisions

Excluded:

Circumventing source restrictions
Contacting competitors
Publishing allegations
Automatic price or positioning changes
Treating rumors as verified facts

4. Domain model

Competitor

Stores category, priority, target segments, monitored sources, status, and description.

CompetitorOffering

Stores target user, delivery model, features, pricing summary, availability, observed date, sources, and confidence.

ResearchSource

Stores source type, URL, publisher, title, publication/observation date, retrieval method, terms status, content hash, snapshot, status, and refresh policy.

CompetitorObservation

Stores observation type, statement, classification, confidence, effective/expiry dates, verification, and source.

MarketChange

Stores old/new observations, summary, change type, materiality, confidence, detected time, and status.

PricePoint

Stores offering, currency, amount, billing period, type, conditions, observation date, source, and confidence.

StrategicAlert

Stores fact, interpretation, possible implication, recommended action, domain, urgency, materiality, confidence, and decision linkage.

MarketBrief

References an artifact and stores period, covered competitors, changes, pricing freshness, top decisions, and approval.

5. Notion workspace

Competitive Intelligence

Projects competitors, material observations, changes, sources, freshness, confidence, implications, and action status.

Pricing Comparison

Projects comparable offers, price points, conditions, evidence dates, and comparability warnings.

Strategic Alerts

Founder may dismiss, watch, create a product signal, create content brief, or open pricing/strategy decision through controlled commands.

6. Agents

fos.market_watcher

fos.competitive_evidence_extractor

fos.market_change_detector

fos.job_based_competitor_comparison

fos.pricing_comparison

fos.strategy_signal

fos.market_brief

The Strategy Signal Agent must output observed fact, interpretation, possible implication, recommended action, and confidence as separate fields.

7. Workflows

Monitoring

Retrieve approved source, verify policy, hash content, skip unchanged content, persist source snapshot/reference, and queue extraction.

Observation and change

Extract facts, label company claims, compare with baseline, suppress formatting-only change, calculate materiality, and create MarketChange.

Strategic alert

Deduplicate, assess relevance to enrollment/product/pricing/communications, project to Notion, and allow founder to open canonical decision or signal.

Pricing review

Compare only reasonably comparable offers, display conditions and dates, avoid false precision, and route any pricing recommendation to Phase 6 decision governance.

8. APIs and jobs

API families:

/api/fos/competitors/*

/api/fos/research-sources/*

/api/fos/competitor-observations/*

/api/fos/market-changes/*

/api/fos/price-points/*

/api/fos/strategic-alerts/*

/api/fos/market-briefs/*

Jobs:

monitor-research-source

extract-competitor-observations

detect-market-change

refresh-pricing-comparison

generate-job-based-comparison

generate-strategic-alert

generate-weekly-market-brief

expire-stale-market-evidence

reconcile-market-projections

9. Deterministic safeguards

Respect source access and terms policies.

Company claims are labeled as company-provided.

Low-confidence rumor cannot create a high-confidence fact.

Stale prices are visibly stale.

Unchanged sources do not generate alerts.

Notion cannot alter evidence or price points directly.

Market findings cannot change roadmap, positioning, or price without founder decision.

10. Tests

Tests cover unchanged source, pricing change, removed feature, company-claim labeling, rumor handling, duplicate alert suppression, stale price, strategic decision linkage, workspace conflict, and malicious webpage injection.

	11. Work packages

Package

Deliverable

WP5.0

Competitor taxonomy and approved-source policy

WP5.1

Competitor, offering, and source registry

WP5.2

Retrieval, hashing, and snapshots

WP5.3

Observation extraction and verification

WP5.4

Material change detection

WP5.5

Pricing and offer intelligence

WP5.6

Job-based comparison

WP5.7

Strategic alerts and market briefs

WP5.8

Notion intelligence workspace and decision commands

WP5.9

Evaluation, freshness, and noise tuning

12. Definition of done

Findings are dated, sourced, classified, and confidence-scored.

Material changes are detected with acceptable duplicate/noise levels.

Pricing freshness and comparability are visible.

Founder can convert an alert into a canonical decision, signal, or content brief.

No market agent changes strategy or pricing automatically.

Shared architectural invariants

The following rules are inherited from revised Phase 0 and are non-negotiable for this phase:

FOS owns canonical state and intelligence. Notion is a founder-facing working environment and projection surface.

External workspace changes are commands, not direct mutations. A Notion status change or button creates a validated WorkspaceCommand.

Artifacts are generic and versioned. Enrollment messages, specifications, LinkedIn posts, Substack papers, reports, and reviews use ArtifactRecord and ArtifactVersion.

Approvals are interface-independent. Approval state is canonical and may be requested from Notion, a native interface, or an API.

Evidence, claims, consent, and product availability are deterministic gates. Models may recommend; they may not waive these controls.

Every projection has an ownership policy. Fields are `canonical_read_only`, `working_copy_editable`, `controlled_command`, or `not_projectable`.

Raw events are immutable. Derived summaries may be superseded but not silently overwrite source history.

Every agent is bounded. Each agent has a versioned objective, input schema, output schema, permitted tools, permitted memory scopes, evaluation policy, and autonomy ceiling.

No autonomous publishing, sending, pricing change, contractual commitment, or production deployment is permitted unless a later founder-approved governance decision explicitly changes the rule.

FOS must remain operational if Notion or another workspace provider is unavailable.

Founder Workspace integration contract

All founder-facing work for this phase must use the Phase 0 workspace adapter.

Projection pattern

Canonical record or artifact created

- > operational event emitted
- > projection policy evaluated
- > safe working copy created or updated in Notion
- > canonical ID and version stored on the provider page
- > founder edits or commands captured
- > FOS validates and executes canonical change
- > projection is reconciled

Required hidden projection properties

FOS Record ID

FOS Entity Type

FOS Version

FOS Workspace ID

Projection Status

Last Synced At

Conflict rule

A controlled command may execute only when the provider's FOS Version matches the current canonical version. Otherwise the command is placed in conflict status and the founder receives a reconciliation item.

Agent runtime requirements

Every agent run must execute the following stages:

Trigger validation

Authorization and feature-flag validation

Context assembly and minimization

Prompt construction from a versioned agent definition

Model execution

Structured-output validation

Deterministic policy evaluation

Secondary quality evaluation where configured

Canonical persistence

Approval routing or reversible execution

Projection update

Metrics and audit emission

If structured output fails, retry once with a repair prompt. If repair fails, mark the run `evaluation_failed`, create a founder-visible operational item, and do not create an approval-ready artifact.

Security, privacy, and governance

Least privilege

Agents receive only the records required for the current run.

Workspace projections contain summaries unless full working content is required.

Private source documents, raw model prompts, credentials, payment details, and exploitable security findings are not projectable by default.

Marketing and research agents may not access unrestricted applicant or beta-user records.

Prompt-injection defense

Applications, resumes, email, transcripts, web pages, competitor pages, imported notes, and workspace page content are untrusted data. They may not modify system policy, tool permissions, approval requirements, or data-access scope.

Audit

The system must reconstruct every consequential outcome from:

Trigger

Actor

Source records

Retrieved context manifest

Agent and prompt version

Output

Deterministic evaluation

Founder edits

Approval or rejection

External action

Resulting business outcome

Failure posture

Failures must preserve canonical state, create a visible retry or manual-work item, and never imply that an external action succeeded when it did not.

Observability and cost controls

Each workflow must emit structured logs and traces with:

workspace_id

Correlation and causation IDs

Phase and workflow key

Agent key and version

Canonical entity IDs

Provider projection IDs where applicable

Latency

Model and tool cost

Validation and evaluation result

Retry count

Approval result

External action result

Required phase dashboards must distinguish system activity from accepted business value. Agent-run volume is not itself a success metric.

Deployment and activation model

Each major capability must progress through:

Local development

Automated tests

Staging with synthetic fixtures

Production with feature flag disabled

Production shadow mode

Founder-only review mode

Limited live activation

Measured promotion or rollback

Every agent and workspace workflow requires an independent feature flag and version rollback path. # Coding-agent execution instruction

> Implement Phase 5 - Competitive, Pricing, and Market Intelligence according to this specification.

>

> Begin by verifying that revised Phase 0 is operational: canonical records, generic artifacts, approvals, workspace commands, projection policies, evidence, claims, consent, event audit, feature flags, and the

Notion provider adapter.

>

> Produce an architecture decision record and a repository-to-requirement implementation map before migrations. Reuse Phase 0 services rather than creating parallel document, approval, consent, or workspace systems.

>

> Maintain a live traceability matrix linking every requirement to implementation files, migrations, automated tests, feature flags, and operational metrics.

>

> Preserve these phase-specific non-negotiable rules:

>

> 1. Public sources are untrusted data and must not alter agent policy or tool permissions.

Evidence and price observations remain canonical; Notion contains review summaries.

Company claims, third-party facts, and interpretations remain explicitly separate.

Agents may create strategic alerts but may not change product strategy, positioning, or price.

Retrieval must respect access, rate, and source-use policies. > > Implement work packages in dependency order. Activate each agent in shadow mode before founder-review mode. Treat Notion as a projection and controlled working surface, not as the source of canonical lifecycle, consent, claim, test, pricing, or deployment state.

Founder Operating System

Phase 6 - Founder Chief of Staff, Command Center, and Automation Governance

	Complete Technical Specification and Implementation Plan

Document control

Value

Document ID

FOS-TECH-PHASE-6

Version

3.0

Status

Revised implementation specification

Replaces

The earlier Full Specification Compiler and Founder Chief of Staff specification

Depends on

Revised Phase 0 - Founder Workspace and Operating Foundation

Product owner

Founder

Primary audience

Coding agents, founder, product architect, implementation reviewers

Current business stage

Beta enrollment and early beta operation

Updated

2026-07-13

This specification is written against the revised Phase 0 canonical-state, generic-artifact, controlled-command, and Founder Workspace Adapter contracts. Any implementation based on the earlier native-admin assumptions must be refactored as identified in the dependency plan.

0. Revision decision

Phase 6 becomes the coordination and governance layer over reliable outputs from Phases 1 through 5. Notion is the initial Founder Command Center. A native administrative dashboard is deferred until volume, privacy, latency, or productization justifies it.

The chief-of-staff agent does not operate the company autonomously. It reduces the founder's decision surface, identifies conflicts, produces operating reviews, and proposes safe automation opportunities.

1. Implementation directive

Build a single canonical decision queue and operating-review system that ranks consequential work, detects cross-domain conflicts, compiles full specifications, and identifies repeated founder work suitable for bounded automation.

2. Objectives and metrics

Reduce founder cognitive load and decision latency.

Limit daily review to consequential items.

Detect conflicts before external impact.

Produce daily and weekly operating reviews.

Create implementation-grade specifications from approved problems.

Identify time-consuming patterns suitable for automation.

Preserve founder authority over strategy, pricing, publishing, deployment, and commitments.

Metrics include decision time, unresolved critical items, recommendation acceptance, duplicate rate, conflicts caught, founder hours saved, automation opportunities implemented, and agent-caused rework.

3. Scope

Included:

Strategic priority registry

Unified decision queue

Founder recommendations

Daily brief

Weekly/monthly/cohort/release operating reviews

Cross-domain conflict detection

Full specification compiler and critic

Automation opportunity detection

Delegation/autonomy proposals

Notion Founder Command Center

Excluded:

Autonomous strategy changes

Autonomous spending or purchasing

Autonomous pricing, publication, deployment, or contractual commitments

Unbounded task creation

Hidden ranking or unexplained priority changes

4. Domain model

StrategicPriority

Stores title, description, rank, effective dates, metrics, non-goals, status, and founder approval.

DecisionQueueItem

Stores domain, type, source, title, summary, impact, urgency, confidence, estimated founder minutes, priority score, recommended action, risk of delay, status, and due date.

FounderRecommendation

Stores recommendation, rationale, evidence, expected impact, effort, confidence, alternatives, risks, and decision linkage.

OperatingReview

References an artifact and stores period, metrics snapshot, shipped work, enrollment drivers, user struggles, funnel changes, founder time, agent failures, market changes, work to stop, automation opportunities, and top decisions.

ConflictRecord

Stores type, entities, description, severity, recommended resolution, status, and resolution audit.

AutomationOpportunity

Stores task pattern, frequency, founder minutes, monthly hours, risk, required data, proposed agent/workflow, implementation effort, value, autonomy ceiling, and status.

AutonomyPolicyProposal

Stores current/proposed autonomy, workflow, evidence, success thresholds, rollback, prohibited actions, founder decision, and effective dates.

5. Notion Founder Command Center

Required views:

Decisions required today

Enrollment and beta risks

Release blockers

Marketing approvals

Strategic alerts

Conflicts

Deferred decisions

Automation opportunities

Weekly operating review

Founder workload and time savings

Every item links to canonical records and exposes ranking factors. Notion commands approve, reject, defer, pin, request analysis, or open a decision; they do not directly mutate protected state.

6. Agents

Founder Chief of Staff - `fos.founder_chief_of_staff`

Produces bounded daily and weekly decision summaries.

Cross-Domain Conflict Detector - `fos.cross_domain_conflict_detector`

Detects enrollment promise versus capability, marketing claim versus release, pricing versus offer, consent versus content, requirement versus test, resource versus priority, and similar contradictions.

Full Specification Compiler - `fos.full_specification_compiler`

Produces strategic context, evidence, alternatives, scope, requirements, agent contracts, data/API/security/observability/migration/testing/rollout/rollback, success metrics, risks, and open questions.

Specification Critic - `fos.specification_critic`

Evaluates customer value, revenue, opportunity cost, architecture, security, privacy, complexity, testability, maintainability, and go-to-market consistency.

Automation Opportunity Detector - `fos.automation_opportunity_detector`

Identifies repeated founder actions and estimates value, risk, data needs, implementation effort, and suitable autonomy ceiling.

Autonomy Governance Evaluator - `fos.autonomy_governance`

Evaluates whether a workflow can move from observe to draft or reversible execution based on measured evidence. It cannot approve the increase.

7. Prioritization model

Default priority uses approved strategic alignment, business impact, urgency, confidence, risk of delay, enrollment value, founder-time saving, implementation effort, and reversibility risk.

Critical security, privacy, legal, or customer-harm items override economic ranking. The system must show ranking factors and permit founder pin, defer, or suppression.

8. Workflows

Daily brief

Collect unresolved consequential items, exclude informational noise, deduplicate, detect conflicts, rank, limit default count, create decision artifacts, and project to Notion.

Weekly operating review

Snapshot enrollment, beta, product, QA, content, market, agent, and founder-time metrics; summarize changes; identify work to stop; propose three highest-value decisions; and require founder review.

Full specification

Start from approved problem, assemble evidence/constraints, generate artifact, run critic, revise, validate requirement/test plan, route approval, and create traceability records.

Conflict scan

Run after product release, claim/price/offer update, content approval, specification approval, consent change, or strategic-priority update.

Automation governance

Detect pattern, calculate founder cost, assess risk, propose bounded workflow, specify metrics and rollback, run shadow mode, and request founder decision before autonomy changes.

9. APIs and jobs

API families:

/api/fos/strategic-priorities/*

/api/fos/decision-queue/*

/api/fos/founder-recommendations/*

/api/fos/operating-reviews/*

/api/fos/conflicts/*

/api/fos/automation-opportunities/*

/api/fos/autonomy-policies/*

/api/fos/specifications/full/*

Jobs:

build-daily-founder-brief

build-weekly-operating-review

scan-cross-domain-conflicts

generate-full-specification

run-specification-critique

detect-automation-opportunities

evaluate-autonomy-policy

reconcile-command-center-projections

10. Deterministic safeguards

Chief-of-staff recommendations cannot create or change strategic priorities.

Informational activity is summarized rather than promoted to decisions.

Daily queue is bounded and duplicate-suppressed.

Ranking inputs are inspectable.

Conflict records cannot be silently resolved.

Autonomy changes require founder approval and rollback.

Native command-center UI is deferred until evidence justifies it.

11. Tests

Decision tests cover revenue versus low-value tasks, security override, confidence demotion, deferral, duplication, ranking explanation, and stale item handling.

Conflict tests cover claim/capability, enrollment/scope, price/offer, requirement/test, consent/content, and release/publication contradictions.

Chief-of-staff tests verify bounded actionable output, no priority mutation, inspectable evidence, and audit of rejected recommendations.

Autonomy tests verify shadow-mode thresholds, rollback, prohibited actions, and founder approval.

Deliverable

Cross-phase data quality and metric readiness audit

Strategic priority registry

WP6.2

Unified canonical decision queue

WP6.3

Notion Founder Command Center

WP6.4

Cross-domain conflict detection

WP6.5

Daily founder brief

WP6.6

Weekly and monthly operating reviews

WP6.7

Full specification compiler and critic

WP6.8

Automation opportunity detection

WP6.9

Autonomy governance and rollback

WP6.10

Evaluation, tuning, and native-UI decision gate

13. Definition of done

Founder operates from one bounded decision queue.

Recommendations are evidence-backed and explainable.

Cross-domain conflicts are detected before protected actions.

Operating reviews show business results, founder time, and agent quality.

Full specifications use shared artifacts and canonical requirements/tests.

Automation and autonomy changes remain founder-governed and reversible.

FOS remains functional without Notion.

Shared architectural invariants

The following rules are inherited from revised Phase 0 and are non-negotiable for this phase:

FOS owns canonical state and intelligence. Notion is a founder-facing working environment and projection surface.

External workspace changes are commands, not direct mutations. A Notion status change or button creates a validated WorkspaceCommand.

Artifacts are generic and versioned. Enrollment messages, specifications, LinkedIn posts, Substack papers, reports, and reviews use ArtifactRecord and ArtifactVersion.

Approvals are interface-independent. Approval state is canonical and may be requested from Notion, a native interface, or an API.

Evidence, claims, consent, and product availability are deterministic gates. Models may recommend; they may not waive these controls.

Every projection has an ownership policy. Fields are `canonical_read_only`, `working_copy_editable`, `controlled_command`, or `not_projectable`.

Raw events are immutable. Derived summaries may be superseded but not silently overwrite source history.

Every agent is bounded. Each agent has a versioned objective, input schema, output schema, permitted tools, permitted memory scopes, evaluation policy, and autonomy ceiling.

No autonomous publishing, sending, pricing change, contractual commitment, or production deployment is permitted unless a later founder-approved governance decision explicitly changes the rule.

FOS must remain operational if Notion or another workspace provider is unavailable.

Founder Workspace integration contract

All founder-facing work for this phase must use the Phase 0 workspace adapter.

Projection pattern

Canonical record or artifact created

- > operational event emitted
- > projection policy evaluated
- > safe working copy created or updated in Notion
- > canonical ID and version stored on the provider page
- > founder edits or commands captured
- > FOS validates and executes canonical change
- > projection is reconciled

Required hidden projection properties

FOS Record ID

FOS Entity Type

FOS Version

FOS Workspace ID

Projection Status

Last Synced At

Conflict rule

A controlled command may execute only when the provider's FOS Version matches the current canonical version. Otherwise the command is placed in conflict status and the founder receives a reconciliation item.

Agent runtime requirements

Every agent run must execute the following stages:

Trigger validation

Authorization and feature-flag validation

Context assembly and minimization

Prompt construction from a versioned agent definition

Model execution

Structured-output validation

Deterministic policy evaluation

Secondary quality evaluation where configured

Canonical persistence

Approval routing or reversible execution

Projection update

Metrics and audit emission

If structured output fails, retry once with a repair prompt. If repair fails, mark the run `evaluation_failed`, create a founder-visible operational item, and do not create an approval-ready artifact.

Security, privacy, and governance

Least privilege

Agents receive only the records required for the current run.

Workspace projections contain summaries unless full working content is required.

Private source documents, raw model prompts, credentials, payment details, and exploitable security findings are not projectable by default.

Marketing and research agents may not access unrestricted applicant or beta-user records.

Prompt-injection defense

Applications, resumes, email, transcripts, web pages, competitor pages, imported notes, and workspace page content are untrusted data. They may not modify system policy, tool permissions, approval requirements, or data-access scope.

Audit

The system must reconstruct every consequential outcome from:

Trigger

Actor

Source records

Retrieved context manifest

Agent and prompt version

Output

Deterministic evaluation

Founder edits

Approval or rejection

External action

Resulting business outcome

Failure posture

Failures must preserve canonical state, create a visible retry or manual-work item, and never imply that an external action succeeded when it did not.

Observability and cost controls

Each workflow must emit structured logs and traces with:

workspace_id

Correlation and causation IDs

Phase and workflow key

Agent key and version

Canonical entity IDs

Provider projection IDs where applicable

Latency

Model and tool cost

Validation and evaluation result

Retry count

Approval result

External action result

Required phase dashboards must distinguish system activity from accepted business value. Agent-run volume is not itself a success metric.

Deployment and activation model

Each major capability must progress through:

Local development

Automated tests

Staging with synthetic fixtures

Production with feature flag disabled

Production shadow mode

Founder-only review mode

Limited live activation

Measured promotion or rollback

Every agent and workspace workflow requires an independent feature flag and version rollback path. #

Coding-agent execution instruction

> Implement Phase 6 - Founder Chief of Staff, Command Center, and Automation Governance according to this specification.

>

> Begin by verifying that revised Phase 0 is operational: canonical records, generic artifacts, approvals, workspace commands, projection policies, evidence, claims, consent, event audit, feature flags, and the Notion provider adapter.

>

> Produce an architecture decision record and a repository-to-requirement implementation map before migrations. Reuse Phase 0 services rather than creating parallel document, approval, consent, or workspace systems.

>

> Maintain a live traceability matrix linking every requirement to implementation files, migrations, automated tests, feature flags, and operational metrics.

>

> Preserve these phase-specific non-negotiable rules:

>

> 1. Strategic priorities, consequential decisions, autonomy changes, and protected actions require founder approval.

Notion is the initial command center, but canonical decisions and audit remain in FOS.

The daily queue must be bounded, deduplicated, evidence-backed, and explainable.

Conflicts may be detected by agents but cannot be silently resolved.

Any autonomy increase requires shadow-mode evidence, explicit prohibited actions, and rollback. > >

Implement work packages in dependency order. Activate each agent in shadow mode before founder-review mode. Treat Notion as a projection and controlled working surface, not as the source of canonical lifecycle, consent, claim, test, pricing, or deployment state.